FLEETX PROJECT REPORT

FleetX — Intelligent Fleet & Logistics Management System

A Project-Based Learning (PBL) Software Engineering Project

Submitted in partial fulfilment of the academic requirements
for the Academic Year 2026

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Under the supervision of
Er. RAM BABU BURI

S. No.	Student Name	Roll Number	Project Responsibility
1	MD ZIYAUL MUSTAFA	24EARAD100	Team Leader & Lead Systems Architect; Module 1 — Identity & Administration
2	MD AMIR ALAM	24EARAD099	Shipment & Route Management Systems Engineer; Module 4
3	KHUSHARG GAUTAM	24EARAD078	Live Tracking & Control Tower Systems Engineer; Module 5
4	JITENDRA KUMAR PASWAN	24EARAD071	Fleet & Asset Management Systems Engineer; Module 2
5	MAYANK KUMAR	24EARAD098	Driver & Trip Operations Systems Engineer; Module 3

Academic Project Documentation • IEEE-Inspired Technical Report Format

CERTIFICATE OF ORIGINALITY AND APPROVAL

This is to certify that the project report entitled "FleetX — Intelligent Fleet & Logistics Management System" is submitted by the following students of the Department of Artificial Intelligence & Data Science for the academic year 2026:

- MD ZIYAUL MUSTAFA (24EARAD100)
- MD AMIR ALAM (24EARAD099)
- KHUSHARG GAUTAM (24EARAD078)
- JITENDRA KUMAR PASWAN (24EARAD071)
- MAYANK KUMAR (24EARAD098)

The work documented herein is presented as a Project-Based Learning (PBL) software engineering project carried out under the guidance of Er. RAM BABU BURI. To the best of the evaluators' knowledge, the report describes the stated architecture, implementation scope, and verification results supplied by the project team. Approval is subject to institutional review and demonstration.

The signatures below are reserved for the authorised faculty and institutional representatives. This template is not a substitute for the institution's official certificate.

Role	**Name / Designation**	**Signature**	**Date**
Project Guide	Er. RAM BABU BURI		
Head of Department	Department of AI & Data Science		
External/Internal Evaluator	As appointed by institution		

STUDENT DECLARATION

We, the undersigned students, declare that the project titled “FleetX — Intelligent Fleet & Logistics Management System” has been prepared for academic evaluation during 2026 under the guidance of Er. RAM BABU BURI. We accept responsibility for the correctness of the technical descriptions and for clearly distinguishing implemented capabilities from simulations, approximations, and proposed future work.

We further declare that sources used in preparing the report are acknowledged in the references, that the reported verification totals reflect the project specifications provided to us, and that no simulated telemetry or estimated route distance is represented as a measurement from physical GPS hardware. Any institutional originality or similarity-check requirements remain subject to the applicable college/university process.

Student	**Roll Number**	**Signature**	**Date**
MD ZIYAUL MUSTAFA	24EARAD100		
MD AMIR ALAM	24EARAD099		
KHUSHARG GAUTAM	24EARAD078		
JITENDRA KUMAR PASWAN	24EARAD071		
MAYANK KUMAR	24EARAD098		

ACKNOWLEDGEMENTS

The project team expresses sincere gratitude to Er. RAM BABU BURI, Project Guide, for academic guidance, technical direction, and support throughout the planning and development of FleetX. The team also acknowledges the Head of the Department of Artificial Intelligence & Data Science, institutional leadership, laboratory and technical staff, and faculty members whose academic environment supports practical software engineering.

The team thanks its peers for discussions, feedback, and collaborative learning. The modular allocation of responsibilities enabled the members to study identity security, fleet asset management, driver operations, shipment logistics, and real-time visualization as related but separately governed domains. Finally, the team acknowledges the maintainers of the technologies and documentation cited in this report.

ABSTRACT

FleetX is an intelligent fleet and logistics management system designed as a decoupled modular monolith for coordinated management of fleet assets, drivers, trips, shipments, and telemetry-oriented operational visibility. The backend is implemented using Java 21 LTS and Spring Boot 3.5.x, with Spring Data JPA, Spring Security, and STOMP messaging over WebSocket/SockJS. MySQL 8.4 LTS provides relational persistence, while Flyway migrations govern schema evolution. The frontend uses React 18, TypeScript, Vite, Bootstrap 5.3, Leaflet with OpenStreetMap for two-dimensional tracking, and Three.js for a stylized three-dimensional fleet view.

The architecture keeps five domain modules in one deployable Spring Boot artifact while enforcing domain boundaries through immutable data-transfer objects (DTOs) and declared lookup/publisher interfaces. It avoids direct cross-domain repository and entity coupling. Functional scope includes session-based authentication and role-based access control, fleet lifecycle and fuel-efficiency calculations, preventive-maintenance alerts, driver-license checks, trip-state transitions, shipment manifests, Haversine-based route-distance estimation, simulated GPS waypoint interpolation, WebSocket broadcasts, and geofence-breach detection.

The supplied verification record reports 88 of 88 backend automated tests passing and 41 of 41 frontend tests passing. Restart persistence was verified against MySQL 8.4 LTS with zero schema drift. The system explicitly identifies telemetry as simulated, uses straight-line Haversine distances rather than road-network routing, and presents the Three.js scene as a stylized visualization rather than a physical-world digital twin. These disclosures delimit the claims made by the project and provide a basis for future integration with physical telemetry, road-network routing, and mobile clients.

Keywords—fleet management, logistics, modular monolith, Spring Boot, role-based access control, Flyway, simulated GPS, WebSocket, Haversine distance, geofencing.

TABLE OF CONTENTS

This report is organised into preliminary pages, eight technical chapters, and IEEE-style references. In Microsoft Word, update the automatic table of contents if one is inserted by the editor; the chapter outline below provides the report navigation.

- Preliminary Pages: Certificate, Declaration, Acknowledgements, Abstract, Acronyms
- Chapter 1 — Introduction & Background
- Chapter 2 — Literature Review & Architectural Comparison
- Chapter 3 — System Requirements & Specifications (SRS)
- Chapter 4 — System Design & Detailed Architecture
- Chapter 5 — Module-Wise Implementation & Algorithms
- Chapter 6 — Team Governance, RACI Matrix & Work Allocation
- Chapter 7 — Testing, Quality Assurance & Verification
- Chapter 8 — Conclusion, Honest Disclosures & Future Work
- References

LIST OF ACRONYMS

Acronym	**Meaning**
API	Application Programming Interface
BCrypt	Adaptive password-hashing function
CSRF	Cross-Site Request Forgery
DTO	Data Transfer Object
ETA	Estimated Time of Arrival
FR	Functional Requirement
GIS	Geographic Information System
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
LTS	Long-Term Support
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
SRS	Software Requirements Specification
STOMP	Simple Text Oriented Messaging Protocol
UI	User Interface
WebSocket	Full-duplex communication protocol over a single connection
XSRF	Cross-Site Request Forgery (token naming convention)

CHAPTER 1: INTRODUCTION & BACKGROUND

1.1 Context and Industrial Need in Logistics and Transportation

Fleet and logistics operations coordinate vehicles, drivers, dispatches, shipments, maintenance, fuel usage, and delivery milestones. When these functions are maintained in separate spreadsheets or disconnected applications, operational information can become fragmented. Dispatch staff may not have a unified view of vehicle availability; maintenance planning may depend on manually checked odometer readings; and shipment progress may be difficult to correlate with a trip or a vehicle.

A fleet information system provides a shared operational model for these entities and the transitions between them. In a production environment, such a system may also integrate telematics hardware, road-network services, driver applications, and enterprise resource planning systems. FleetX focuses on the software engineering foundations of this coordination: domain separation, controlled state transitions, persistent data, access control, and a live-style operational view.

1.2 Motivation for FleetX

The primary motivation is to demonstrate how a multi-domain enterprise application can remain maintainable without immediately adopting distributed microservices. FleetX brings five closely related domains into one deployable application while keeping their code and persistence responsibilities isolated. This supports a manageable collegiate development and deployment process while retaining explicit contracts between modules.

A second motivation is traceability. Database schema changes are managed through Flyway migrations rather than implicit runtime schema creation, and Hibernate is configured with `ddl-auto=validate`. This makes the expected schema an explicit, version-controlled part of the application. The system also uses session security, CSRF protection, and BCrypt password hashing as core application concerns.

1.3 Problem Statements

- Operational silos: fleet assets, driver records, trips, and shipments may be maintained independently, complicating cross-domain coordination.
- Maintenance blindspots: odometer and fuel information may not be consistently converted into efficiency indicators or threshold-based maintenance alerts.
- Dispatch bottlenecks: assigning a vehicle and driver requires validation of availability and driver compliance; informal processes can permit invalid assignments.
- Visibility vacuum: dispatch and logistics teams need a coherent view of trip progress and delivery milestones, but physical telemetry is not always available in a student project environment.
- Schema uncertainty: uncontrolled database changes can create environment-specific differences and make restart behaviour difficult to verify.
- Boundary erosion: direct access to another module’s repositories or entities can create tight coupling that makes individual modules harder to test and change.

1.4 Project Objectives

The objectives are framed as verifiable engineering outcomes. Where a target is not backed by a supplied measurement, it is identified as a design objective rather than a proven performance result.

ID	Objective	Verification approach
OBJ-1	Implement five domain modules in a decoupled modular monolith.	Inspect module packages and inter-module contracts; prohibit direct cross-domain repository/entity coupling.
OBJ-2	Manage database evolution through Flyway migrations and validate the schema at startup.	Execute migration scripts and validate the schema using Hibernate `ddl-auto=validate`.
OBJ-3	Provide operational visibility through simulated GPS waypoints and STOMP/WebSocket broadcasts.	Verify telemetry events reach the frontend and are explicitly labelled as simulated.
OBJ-4	Support fleet, driver, trip, and shipment lifecycle	Exercise documented state transitions and

	workflows.	validation rules through automated tests.
OBJ-5	Apply session-based authentication, RBAC, CSRF protection, and BCrypt password hashing.	Test protected endpoints, CSRF rejection, session cookie attributes, and password-hash handling.
OBJ-6	Establish an automated regression baseline.	Use the supplied test totals: 88/88 backend and 41/41 frontend tests passing.

1.5 Organization of the Report

Chapter 1 introduces the problem and objectives. Chapter 2 compares architectural options and database governance. Chapter 3 defines stakeholders and requirements. Chapter 4 describes system boundaries, persistence, security, and telemetry architecture. Chapter 5 explains each module and its principal algorithms. Chapter 6 documents ownership and collaboration controls. Chapter 7 presents the supplied verification record and a reproducible QA framework. Chapter 8 summarizes outcomes, limitations, disclosures, and future work.

CHAPTER 2: LITERATURE REVIEW & ARCHITECTURAL COMPARISON

2.1 Existing Fleet Management Systems and Commercial Solutions

Commercial fleet-management platforms commonly combine vehicle records, maintenance schedules, driver information, dispatch, location tracking, fuel reporting, and delivery status. The exact feature set varies by provider and subscription. FleetX is not presented as a commercial competitor benchmark or as a replacement for production telematics: its scope is an academic implementation of the underlying information-management patterns.

A useful conceptual comparison is between spreadsheet-led operations, integrated fleet platforms, and FleetX. Spreadsheets are easy to start but provide limited enforcement of workflow invariants and concurrent updates. Commercial platforms may offer live hardware feeds, routing engines, mobile applications, and service-level guarantees, but those capabilities are outside the confirmed scope of this project. FleetX demonstrates modular domain workflows and simulated telemetry with explicit limitations.

Dimension	Spreadsheet / isolated tools	Commercial fleet platform (general pattern)	FleetX
Data integration	Manual reconciliation often required	Integrated data model and vendor connectors may be available	Five related domains in one application
Lifecycle controls	Usually informal	Workflow controls depend on product	Explicit trip and shipment state models
Location	Manual or separate tracking tools	May use physical telematics/GPS	Software-simulated waypoint interpolation
Route distance	Manual estimate or external tool	May use road graph and traffic data	Straight-line Haversine estimate
Schema control	Files and local formats	Vendor-managed	MySQL 8.4 LTS with Flyway migrations
Visualisation	Static sheets/charts	Vendor-specific dashboards	Leaflet 2D and stylized Three.js 3D view

2.2 Architectural Trade-Off Analysis

Three patterns are relevant to FleetX: a tightly coupled monolith, a distributed microservice system, and a modular monolith. A tightly coupled or “spaghetti” monolith can be straightforward to deploy initially, but internal dependencies may spread across modules and make changes risky. Microservices isolate deployment and scaling units, but introduce network calls, service discovery, distributed observability, independent deployment, and cross-service consistency concerns. A modular monolith seeks an intermediate point: one deployable artifact with explicit internal boundaries.

Criterion	Tightly coupled monolith	Distributed microservices	FleetX modular monolith
Deployment	Single artifact	Multiple services	Single Spring Boot artifact

Internal boundaries	Often informal	Process/network boundaries	Compiler-enforced packages/contracts by design
Operational overhead	Low initially	Higher: orchestration and distributed operations	Moderate and suitable for one application deployment
Inter-domain calls	Direct object/repository access can spread	Network APIs or messaging	Immutable DTOs and declared lookup interfaces
Independent scaling	Whole application	Per-service scaling possible	Whole application scales together
Data consistency	Often simple shared transaction boundary	Requires distributed design where data crosses services	Relational persistence with domain boundaries in code
Best fit	Small systems with weak domain separation	Large systems with independent ownership and operations	Related domains with one deployment and strict code boundaries

2.3 Justification for the Decoupled Modular Monolith

FleetX's five domains share a common operational vocabulary—vehicles, drivers, trips, shipments, telemetry, and alerts. Their workflows are related, but they have distinct ownership and schema namespaces. A modular monolith supports one deployment and one relational database while making those ownership boundaries explicit. It reduces the operational burden of running a distributed system in an academic environment without accepting uncontrolled coupling as the default.

The boundary policy is specific: modules exchange immutable DTOs and use declared interfaces such as `VehicleLookupService`, `DriverLookupService`, `TripLookupService`, `ShipmentLookupService`, and `AlertPublisher`. A module must not directly read another domain's JPA repository or depend on another domain's persistence entity. This policy supports replaceable implementations and focused tests. It does not make the modules separately deployable; that is a deliberate trade-off.

2.4 Database Governance: ORM DDL vs. Flyway

Automatic ORM schema generation can be convenient during prototypes, but it can also make database changes implicit and dependent on runtime mappings. In contrast, Flyway migration scripts provide ordered, version-controlled changes that can be reviewed and reproduced across environments. FleetX uses ten migrations spanning V100 through V501 and creates 21 relational tables in MySQL InnoDB with UTF-8 MB4. Hibernate's `ddl-auto` setting is locked to `validate`, so Hibernate checks mapping/schema compatibility without automatically mutating the schema.

Concern	**Automatic ORM DDL generation**	**Flyway + Hibernate validate**
Change history	May be implicit or environment-dependent	Explicit versioned migration files
Review	DDL may be generated at runtime	Migration scripts can be code-reviewed
Startup behaviour	Can create/update schema if configured	Flyway applies migrations; Hibernate validates
Reproducibility	Depends on mapping and configuration	Ordered migration history
FleetX choice	Not used for schema mutation	Adopted

CHAPTER 3: SYSTEM REQUIREMENTS & SPECIFICATIONS (SRS)

3.1 Stakeholder Personas and RBAC Matrix

The following matrix defines a recommended role interpretation for the stated roles. Exact endpoint-to-role annotations must be verified against the implementation before this matrix is used as a security test oracle.

Capability	**ADMIN**	**FLEET_MANAGER**	**DISPATCHER**	**DRIVER**	**VIEWER**
Manage users, roles, and administrative settings	Full	No	No	No	No
View and manage fleet assets / maintenance	Full	Manage	View	Assigned context only	Read-only

Manage driver registry and compliance	Full	Manage	View/validate for dispatch	Own profile / own documents	Read-only
Create and assign trips	Full	Manage	Manage	View assigned trip	Read-only
Manage shipment manifests and milestones	Full	Manage	Operational updates as authorised	Assigned shipment context	Read-only
View live/simulated telemetry and maps	Full	Full	Full	Assigned trip context	Read-only
View administrative audit records	Full	Restricted if granted	No	No	No

Least privilege is the governing principle. ADMIN is intended for identity and system administration; FLEET_MANAGER manages assets and operational configuration; DISPATCHER coordinates driver/vehicle assignment and trips; DRIVER accesses only relevant assigned work; VIEWER receives read-only operational visibility. The matrix is a design-level interpretation of the requested personas, not a claim that every listed permission has been independently audited.

3.2 Functional Requirements

Module 1 — Identity, RBAC, Session Security and Audit

- FR-101: The system shall authenticate users and maintain authenticated state through an HTTP session identified by `JSESSIONID`.
- FR-102: The system shall enforce role-based access decisions for protected operations.
- FR-103: The system shall use BCrypt to hash passwords rather than store plaintext passwords.
- FR-104: The system shall issue and validate CSRF tokens using the configured `XSRF-TOKEN` repository lifecycle.
- FR-105: The system shall support idempotent administrative bootstrap behaviour so repeated startup does not create duplicate bootstrap state.
- FR-106: The system shall record relevant administrative actions in an audit log.

Module 2 — Fleet Asset Lifecycle

- FR-201: The system shall maintain vehicle asset records and lifecycle/status information.
- FR-202: The system shall record odometer and fuel-consumption data needed for efficiency calculation.
- FR-203: The system shall calculate fuel efficiency as odometer delta divided by fuel volume consumed, using km/L.
- FR-204: The system shall evaluate preventive-maintenance thresholds and generate alerts when configured conditions are met.
- FR-205: The system shall expose vehicle lookup information through a declared interface rather than another module’s repository.

Module 3 — Driver Registry and Trip Operations

- FR-301: The system shall maintain commercial driver records and licence validity information.
- FR-302: The system shall check licence expiry before permitting dispatch where the workflow requires a valid licence.
- FR-303: The system shall validate driver and vehicle suitability/availability before assignment.
- FR-304: The system shall enforce the trip state transitions DRAFT → ASSIGNED → IN_TRANSIT → COMPLETED, with cancellation represented by CANCELLED.
- FR-305: The system shall expose trip lookup through `TripLookupService`.

Module 4 — Shipment Manifests and Route Logistics

- FR-401: The system shall maintain shipment manifests and cargo-related records.
- FR-402: The system shall associate shipments with authorised operational trips where applicable.
- FR-403: The system shall estimate great-circle distance using the Haversine formula with Earth radius R = 6371 km.
- FR-404: The system shall track shipment milestones through ORDERED → DISPATCHED → IN_TRANSIT → DELIVERED, with RETURNED as an alternate terminal state.

- FR-405: The system shall expose shipment lookup information through `ShipmentLookupService`.

Module 5 — Telemetry, Control Tower and Visualisation

- FR-501: The system shall generate simulated GPS positions using software waypoint interpolation.
- FR-502: Telemetry responses and user-interface elements shall explicitly identify simulated data.
- FR-503: The system shall publish telemetry updates through Spring WebSocket using STOMP over SockJS.
- FR-504: The frontend shall render two-dimensional map locations and route polylines with Leaflet/OpenStreetMap.
- FR-505: The frontend shall provide a stylized Three.js visualisation synchronized with telemetry state.
- FR-506: The system shall detect circular or polygonal geofence breaches according to configured geofence geometry.

3.3 Non-Functional Requirements

ID	Requirement	Verification / status
NFR-01	Security: authenticated and role-authorised access to protected actions.	Verify access-control tests and endpoint rules.
NFR-02	Session cookies should use HttpOnly and appropriate SameSite settings (Lax/Strict as configured).	Inspect response cookies in integration/security tests.
NFR-03	State-changing requests should be protected by CSRF tokens.	Test missing/invalid token rejection.
NFR-04	Password storage should use BCrypt hashes.	Inspect authentication configuration and persisted representation.
NFR-05	Database schema changes must be version-controlled and reproducible.	Flyway history and startup validation.
NFR-06	Module boundaries must prevent direct cross-domain entity/repository coupling.	Code review, package rules, and architecture tests if present.
NFR-07	Telemetry should support near-real-time UI updates; the requested target is under 500 ms.	Treat as a target until measured under a documented load and network setup.
NFR-08	Backend and frontend regressions should be covered by automated tests.	Supplied results: 88/88 backend and 41/41 frontend passing.
NFR-09	Browser support should cover modern browsers capable of ES modules, WebSocket, and WebGL.	Confirm on the project's target browser matrix before release.
NFR-10	Persistent state should survive application restart without schema drift.	Restart persistence reported verified on MySQL 8.4 LTS.

3.4 Hardware, Software and Network Prerequisites

The following are practical baseline recommendations for development and demonstration, not measured minimums or production sizing guarantees.

Category	Recommended baseline / required component
Development workstation	64-bit system; 8 GB RAM minimum practical baseline, 16 GB recommended; sufficient disk for JDK, Node tooling, dependencies, and MySQL data.
Backend	Java 21 LTS; Spring Boot 3.5.x; Maven wrapper (`./mvnw`) and required dependencies.
Database	MySQL 8.4 LTS; InnoDB; UTF-8 MB4; permissions to create schema and apply migrations.
Frontend	Node.js version compatible with the project's Vite version; npm or the package manager fixed by the repository; modern browser.
Network	HTTP access between browser and backend; WebSocket/SockJS connectivity; access to OpenStreetMap tiles where online map tiles are

	used.
Security configuration	Configured secrets and credentials outside source control; correct origin/CORS and cookie settings for the chosen deployment.

CHAPTER 4: SYSTEM DESIGN & DETAILED ARCHITECTURE

4.1 High-Level Architecture

FleetX is a decoupled modular monolith: the five domain modules are compiled and deployed as one Spring Boot application. The shared common foundation contains reusable contracts and cross-cutting utilities. Persistent data resides in MySQL 8.4 LTS and schema evolution is handled by Flyway.

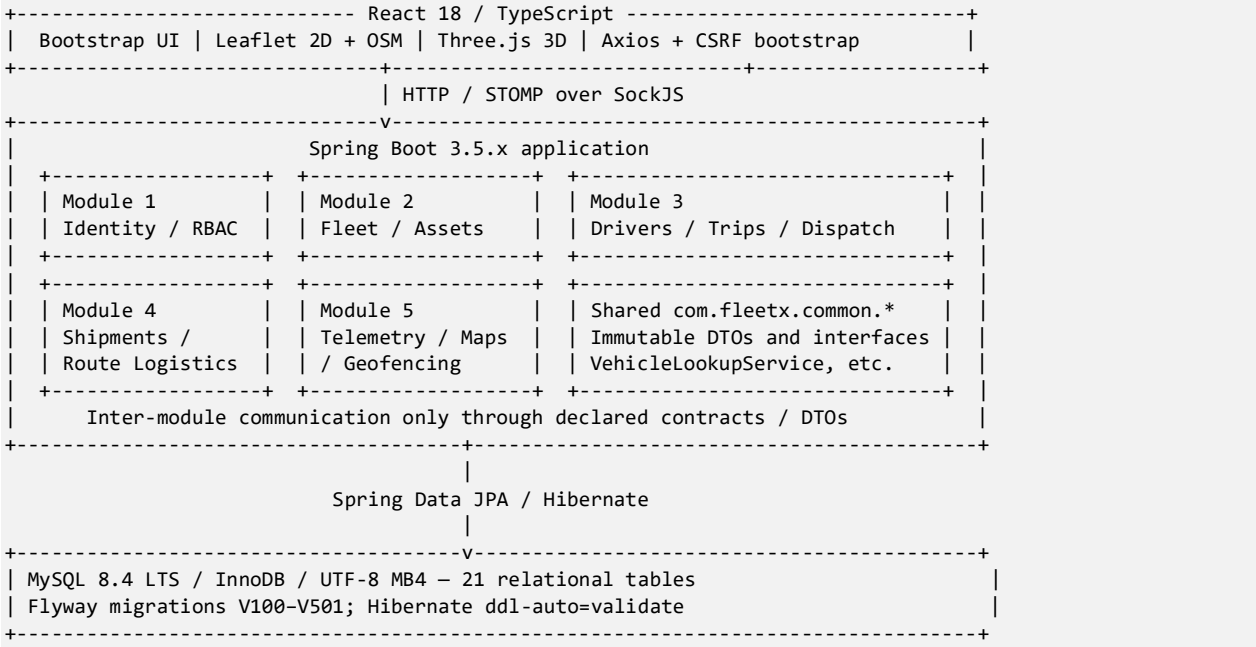


Figure 1. Logical architecture of FleetX (conceptual; package-level names beyond the stated contracts are intentionally not invented).

4.2 Inter-Module Communication and Boundary Enforcement

The modular monolith is governed by explicit ownership. A module may request a small piece of information from another domain through a declared lookup interface. For example, dispatch validation can consult vehicle and driver eligibility through `VehicleLookupService` and `DriverLookupService`, without importing the underlying vehicle or driver persistence entities. An alerting workflow can publish through `AlertPublisher` rather than reaching into another module's repository.

Contract	Intended responsibility	Boundary benefit
VehicleLookupService	Provide the vehicle facts needed by an authorised workflow.	Prevents direct access to fleet persistence internals.
DriverLookupService	Provide driver/licence facts needed for assignment validation.	Keeps driver entity schema private to its owner.
TripLookupService	Expose trip lookup information to dependent workflows.	Avoids direct trip repository coupling.
ShipmentLookupService	Expose shipment lookup information to dependent workflows.	Preserves shipment-domain ownership.
AlertPublisher	Publish alert intent to the responsible alert/notification path.	Reduces direct notification implementation coupling.

Immutable DTOs	Carry only contract-approved data between modules.	Avoid shared mutable entity graphs and lazy-loading leakage.

Boundary enforcement should be supported by code review and, where included in the repository, automated architecture tests. The stated ground truth prohibits direct cross-domain repository or entity coupling. If a future team extracts a module into a separate service, the contract interfaces provide a starting point for an API boundary, but extraction would still require explicit decisions about network failure, data ownership, and transaction semantics.

4.3 Database Schema Design and ER Modelling

The supplied database design contains 21 relational tables in MySQL 8.4 LTS using InnoDB and UTF-8 MB4. The project uses ten Flyway migrations across version identifiers V100 through V501. Module ownership is partitioned by migration namespace: V100–V199 for identity/common administration, V200–V299 for fleet assets, V300–V399 for drivers and trips, V400–V499 for shipments/logistics, and V500–V599 for telemetry-related persistence. The final stated migration identifier is V501.

Because the exact table names and column definitions were not included in the provided specification, the report does not invent a complete physical ERD. The following is a logical entity relationship overview and should be reconciled with the actual migration files before being used as a schema-level diagram.

```
User / Role --------< UserRole
    |
    +---------------< AdminAuditEvent

Vehicle ------------< MaintenanceRecord
   |                 < FuelRecord / OdometerRecord
   |
   +----------------< Trip >---------------- Driver
                        |
                        +------------------< Shipment / Manifest
                        |
                        +------------------< TelemetryWaypoint
Geofence ------------< GeofenceEvent
Alert / Notification records relate to the operational event that generated them.
```

Figure 2. Logical relationship sketch only; actual table names and cardinalities must be confirmed from Flyway migrations.

Schema governance has three steps: (1) author a versioned migration in the owner’s namespace; (2) apply it through Flyway in the intended sequence; and (3) validate the resulting schema against JPA mappings at startup. The `ddl-auto=validate` setting prevents Hibernate from silently repairing a mismatch. A mismatch should be corrected through a reviewed migration or mapping change, not by relying on automatic DDL mutation.

4.4 Security Architecture

4.4.1 Stateful Session Management

After successful authentication, the server maintains authenticated state in an HTTP session identified by `JSESSIONID`. Session cookies are configured as HttpOnly and use SameSite=Lax or Strict according to the application configuration. HttpOnly reduces direct script access to the cookie; SameSite provides an additional cross-site request control. These settings complement, but do not replace, HTTPS, secure session lifecycle management, and authorization checks.

4.4.2 CSRF Token Lifecycle

The frontend bootstraps the CSRF token and submits it with state-changing requests through the configured Axios client. The token repository uses the `XSRF-TOKEN` naming convention. The server validates the token before accepting protected state-changing operations. Security tests should verify that missing or invalid tokens are rejected and that the intended client can obtain and submit the token successfully.

```
Conceptual request lifecycle (not a complete configuration listing):
1. Browser requests application/bootstrap endpoint.
2. Server establishes or associates the HTTP session and CSRF token.
3. Client reads the token according to configured cookie/header policy.
```

```
4. Axios includes the CSRF token on unsafe HTTP methods.
5. Security filter validates the token before controller execution.
```

4.4.3 Password Hashing and Audit

BCrypt is used for password hashing. Passwords should not be stored in plaintext or reversibly encrypted. BCrypt hashes are verified during authentication using the configured password encoder. Administrative actions are recorded through audit logging to support accountability and incident review. Audit records should capture only the data needed for accountability and should not contain passwords, CSRF tokens, or other secrets.

4.5 Real-Time Telemetry Pipeline

The telemetry pipeline begins with `SimulatedGpsService`, which interpolates software-generated positions along configured waypoints. Updates are published through Spring WebSocket with the STOMP sub-protocol over SockJS. The React client consumes updates and refreshes Leaflet markers/polylines and the stylized Three.js fleet canvas. The UI and API explicitly label these positions as simulated data.

```
SimulatedGpsService
        |
        v
Telemetry update / DTO
        |
        v
Spring WebSocket + STOMP over SockJS
        |
        +---------------------> React telemetry state
                                        |
                          +-------------+-------------+
                          |                           |
                          v                           v
                 Leaflet 1.9 (2D)             Three.js r160+ (3D)
                 markers / polylines           stylized fleet scene
```

The requested target of less than 500 ms for STOMP broadcasting is a performance target, not a universal guarantee. A defensible measurement requires a defined test environment, event rate, number of connected clients, network conditions, and latency measurement points. The 2D map uses OpenStreetMap tiles, subject to the relevant tile-service usage policies. The 3D canvas is a procedural visual representation synchronized with telemetry state; it is not a surveyed physical model.

CHAPTER 5: MODULE-WISE IMPLEMENTATION & ALGORITHMS

5.1 Module 1: Identity and System Administration

Author: MD ZIYAUL MUSTAFA (24EARAD100) — Team Leader and Lead Systems Architect.

Module 1 owns identity, role-based access control, session security, administrative bootstrap, audit logging, and the shared common foundation under `com.fleetx.common.*`. Its Flyway namespace is V100–V199.

5.1.1 Authentication and Authorization Workflow

1. The user submits credentials through the client.
2. The server verifies the password using BCrypt and establishes authenticated session state.
3. The browser retains the session identifier in the configured `JSESSIONID` cookie.
4. Each protected request is checked for authentication and the required role/authority.
5. Unsafe state-changing requests are validated against the CSRF token lifecycle.
6. Relevant administrative actions are written to the audit trail.

5.1.2 Idempotent Administrative Bootstrap

An idempotent bootstrapper is designed so that repeated application starts do not repeatedly create duplicate initial administrative records. The bootstrap operation should check whether the required initial state already exists and should avoid embedding a reusable plaintext credential in source control. The supplied specification confirms the presence of an idempotent admin bootstrapper but does not state the exact provisioning mechanism or credential policy.

5.1.3 Shared Contracts

The shared common package should contain only cross-cutting types and stable contracts. Domain-specific JPA entities should remain in their owning module. This reduces the risk that a shared package becomes a second, uncontrolled domain model.

5.2 Module 2: Fleet and Asset Lifecycle Management

Author: JITENDRA KUMAR PASWAN (24EARAD071) — Fleet & Asset Management Systems Engineer.

Module 2 owns vehicle lifecycle information, fuel-efficiency calculations, and preventive-maintenance threshold alerts. Its Flyway namespace is V200–V299.

5.2.1 Vehicle Status and Lifecycle

The fleet domain maintains vehicle records and operational status. Status changes should be validated against the application's lifecycle rules and checked before dispatch. The module exposes only the vehicle information needed by other domains through `VehicleLookupService`; dispatch logic must not query fleet repositories directly.

5.2.2 Fuel Efficiency Calculation

Fuel efficiency is calculated from the change in odometer reading divided by the fuel volume consumed:

$$\text{Efficiency (km/L)} = \Delta\text{Odometer (km)} / \text{Fuel Volume Consumed (L)}$$

where ΔOdometer = current odometer − previous odometer. The calculation is meaningful only when the odometer delta and fuel volume are valid and refer to a compatible measurement interval. The implementation should reject or flag zero/negative fuel volumes, negative odometer deltas, and missing readings rather than producing misleading values.

```
deltaKm = currentOdometerKm - previousOdometerKm
if deltaKm < 0:
    flag invalid odometer sequence
else if fuelLitres <= 0:
    flag invalid fuel volume
else:
    efficiencyKmPerLitre = deltaKm / fuelLitres
```

This pseudocode describes the validation logic conceptually; it is not a verbatim source-code extract.

5.2.3 Preventive-Maintenance Threshold Engine

The maintenance engine evaluates configured thresholds such as a due odometer interval or a maintenance date. When the threshold condition is met, it generates an alert. The precise threshold values and alert escalation policy were not supplied and therefore are not stated as fixed values in this report.

```
for each vehicle:
    determine the applicable maintenance rule
    compare current vehicle measurements/date with the rule
    if the rule is due or overdue:
        publish a maintenance alert through the declared alert contract
```

5.3 Module 3: Commercial Driver and Trip Operations

Author: MAYANK KUMAR (24EARAD098) — Driver & Trip Operations Systems Engineer.

Module 3 owns commercial driver registration, licence verification, trip dispatch, and trip-state management. Its Flyway namespace is V300–V399.

5.3.1 Licence Expiry and Dispatch Validation

Before assignment, the dispatch workflow checks driver licence validity and the relevant driver/vehicle availability conditions. An expired licence should block dispatch when licence validity is a mandatory rule. Validation should occur on the server, even if the frontend also disables an invalid choice, because client-side checks are not an authorization boundary.

5.3.2 Trip Finite State Machine

```
DRAFT ----assign----> ASSIGNED ----start----> IN_TRANSIT ----complete----> COMPLETED
  |                      |                       |
  +------cancel---------->+------cancel---------->+------cancel----------> CANCELLED
```

Figure 3. Trip lifecycle. The permitted cancellation transitions should match the implementation's transition guard rules.

The state machine prevents arbitrary jumps between lifecycle states. For example, a trip should not be marked COMPLETED before it has entered IN_TRANSIT unless the business rules explicitly allow an exception. Each transition should validate the current state, required assignment data, and the actor's permission. The exact terminal-state policy should be kept in one authoritative domain service rather than duplicated across UI components.

5.3.3 Conceptual Transition Guard

```
transition(currentState, requestedAction):
    verify actor is authorised
    verify requestedAction is valid from currentState
    verify required driver, vehicle, and compliance conditions
    persist the next state
    publish relevant event/alert if required
```

5.4 Module 4: Shipment Manifests and Route Logistics

Author: MD AMIR ALAM (24EARAD099) — Shipment & Route Management Systems Engineer.

Module 4 owns shipment manifests, route waypoints, delivery milestones, and Haversine distance estimates. Its Flyway namespace is V400–V499.

5.4.1 Shipment Manifest Workflow

A manifest associates cargo/shipment information with the operational context required to track dispatch and delivery. Shipment milestones provide an auditable progression from order to dispatch, transit, and delivery or return. The module publishes or serves shipment lookup data through `ShipmentLookupService` rather than exposing persistence entities.

5.4.2 Haversine Great-Circle Distance

The Haversine formula estimates the great-circle distance between two points on a sphere. Let the two points have latitudes φ_1 and φ_2 and longitudes λ_1 and λ_2, expressed in radians. Define $\Delta\varphi = \varphi_2 - \varphi_1$ and $\Delta\lambda = \lambda_2 - \lambda_1$. Then:

$$\mathbf{a = \sin^2(\Delta\varphi/2) + \cos(\varphi_1)\cos(\varphi_2)\sin^2(\Delta\lambda/2)}$$

$$\mathbf{d = 2R \cdot \arcsin(\sqrt{a}), \quad R = 6371\ km}$$

The formula uses the central angle between the points and multiplies it by the assumed Earth radius. Degree values must be converted to radians before trigonometric operations. Floating-point rounding can make a slightly exceed the interval [0, 1], so a robust implementation may clamp a to that interval before computing the square root.

```
function haversineKm(lat1Deg, lon1Deg, lat2Deg, lon2Deg):
    toRad = PI / 180
    phi1 = lat1Deg * toRad
    phi2 = lat2Deg * toRad
    dPhi = (lat2Deg - lat1Deg) * toRad
    dLambda = (lon2Deg - lon1Deg) * toRad
    a = sin(dPhi / 2)^2 + cos(phi1) * cos(phi2) * sin(dLambda / 2)^2
    a = min(1, max(0, a))
    return 2 * 6371 * asin(sqrt(a))
```

This distance is a straight-line spherical estimate, not a drivable-road distance. It does not account for road geometry, turn restrictions, traffic, terrain, ferry routes, or access restrictions. Consequently, any ETA derived from it is an estimate whose quality depends on the assumed speed and operating conditions; it should not be presented as a routing-engine result.

5.4.3 Shipment Finite State Machine

```
ORDERED ----dispatch----> DISPATCHED ----begin transit----> IN_TRANSIT
    |                        |                                |
    +------------------------+--------------------------------+--> RETURNED
                                                          IN_TRANSIT --> DELIVERED
```

Figure 4. Shipment lifecycle. Exact permitted return transitions and guards must be consistent with the implementation.

5.5 Module 5: Real-Time Telemetry, Control Tower and Visualisation

Author: KHUSHARG GAUTAM (24EARAD078) — Live Tracking & Control Tower Systems Engineer.

Module 5 owns simulated GPS interpolation, STOMP/WebSocket delivery, 2D map tracking, the Three.js visualisation, and geofencing. Its Flyway namespace is V500–V599.

5.5.1 Software GPS Waypoint Interpolation

The system uses `SimulatedGpsService` to interpolate positions between software-defined waypoints. For a simple linear interpolation between coordinates A and B, a parameter t between 0 and 1 produces an intermediate point $P(t) = (1 - t)A + tB$. This is a coordinate interpolation method, not a claim that a vehicle follows a physically valid road segment. For long distances or high-latitude paths, linear latitude/longitude interpolation is not equivalent to geodesic interpolation.

```
t = clamp(elapsed / segmentDuration, 0, 1)
lat = (1 - t) * startLat + t * endLat
lon = (1 - t) * startLon + t * endLon
publish position with explicit SIMULATED DATA marker
```

5.5.2 STOMP/WebSocket Broadcasting

The backend broadcasts telemetry updates through Spring WebSocket using STOMP over SockJS. The frontend subscribes to the designated message destination and updates its client-side telemetry state. The architecture requests sub-500 ms broadcast latency; this should be validated with timestamps captured at event generation and client receipt, and should report median and tail percentiles under documented load.

5.5.3 Leaflet 2D and Three.js 3D

Leaflet 1.9 renders a two-dimensional map with markers and polylines. OpenStreetMap provides the map data tiles. The Three.js r160+ canvas offers a stylized procedural view of fleet assets and synchronizes visual status with telemetry state. The two views serve different purposes: Leaflet supplies geographic context, while the Three.js canvas supplies an at-a-glance visual status representation.

5.5.4 Geofencing

Geofencing determines whether a reported coordinate lies within or outside a configured geographic region. FleetX supports circular and polygonal breach detection. A circular geofence can compare the Haversine distance from a point to the geofence centre against the configured radius. A polygonal geofence can use a point-in-polygon algorithm in a suitable coordinate plane. Boundary behaviour—whether a point exactly on the boundary counts as inside—should be specified and tested consistently.

```
Circular geofence:
distanceKm = haversineKm(point, centre)
breach = distanceKm > radiusKm

Polygonal geofence:
inside = pointInPolygon(point, polygonVertices)
breach = not inside
```

Geofence results depend on the coordinate source. Because FleetX telemetry is simulated, geofence alerts validate the software workflow rather than prove physical vehicle location.

CHAPTER 6: TEAM GOVERNANCE, RACI MATRIX & WORK ALLOCATION

6.1 Team Ownership Matrix

Member	Role / domain	Migration namespace	Ownership boundary
MD ZIYAUL MUSTAFA (24EARAD100)	Team Leader; Module 1 identity, RBAC, session security, admin audit; shared	V100–V199	`com.fleetx.common.*` contracts and identity/admin domain

	common foundation		
MD AMIR ALAM (24EARAD099)	Module 4 shipment manifests, route waypoints, delivery milestones	V400–V499	Shipment and route-logistics logic
KHUSHARG GAUTAM (24EARAD078)	Module 5 simulated GPS, STOMP, Leaflet, Three.js, geofencing	V500–V599	Live tracking and control-tower workflows
JITENDRA KUMAR PASWAN (24EARAD071)	Module 2 fleet assets, fuel efficiency, preventive maintenance	V200–V299	Fleet/asset records and maintenance logic
MAYANK KUMAR (24EARAD098)	Module 3 driver registry, licence checks, trip dispatch/state machine	V300–V399	Driver and trip operations

6.2 RACI Matrix

R = Responsible for execution; A = Accountable for final delivery; C = Consulted; I = Informed. This proposed RACI matrix translates the supplied ownership into a practical governance model; it should be adjusted if the team's actual review process differs.

Deliverable	Ziyaul Mustafa	Jitendra Kumar	Mayank Kumar	Amir Alam	Khusharg Gautam
Architecture and common contracts	A/R	C	C	C	C
Identity, RBAC and audit	A/R	I	C	I	C
Fleet assets and maintenance	C	A/R	C	I	C
Driver registry and trip dispatch	C	C	A/R	C	C
Shipment manifests and Haversine estimate	C	I	C	A/R	C
Telemetry and visualisation	C	C	C	C	A/R
Cross-module integration	A	R	R	R	R
Backend regression testing	A	R	R	R	R
Frontend regression testing	C	I	I	C	A/R
Final report and demonstration	A	R	R	R	R

6.3 Git Branching, Code Review and Conflict Resolution

The requested minimum of two approvals is a proposed merge policy for changes affecting shared or protected branches. A practical workflow is:

7. Create a short-lived feature branch named for the module and work item.
8. Keep commits focused; do not combine unrelated domain changes in one pull request.
9. Run backend tests (`./mvnw test`), frontend tests, formatting/lint checks configured by the repository, and migration validation before requesting review.
10. Open a pull request with purpose, affected module, migration range, test evidence, and any API/DTO changes.

11. Require at least two approvals for protected-branch merges, including review of boundary violations and schema ownership where applicable.
12. Resolve conflicts by identifying the owning module and the authoritative migration order; do not silently rewrite another module's migration or entity.
13. After merge, run integration tests and verify application startup against the migrated MySQL schema.

For a conflict involving a shared DTO or declared interface, the team lead coordinates a backward-compatible contract decision and informs every affected module owner. For a database conflict, the migration owner and lead agree on a new versioned migration rather than editing an already-applied migration in a shared environment. The precise branching model (for example, GitHub Flow) is a team-process recommendation, not a claim that a specific hosting platform is used.

CHAPTER 7: TESTING, QUALITY ASSURANCE & VERIFICATION

7.1 Backend Automated Testing

The supplied verification record reports 88 of 88 backend tests passing under `./mvnw test`. The named test technologies are JUnit 5, Mockito, AssertJ, Flyway schema validation, and Spring Boot Test. The specification describes coverage across unit, contract, and end-to-end integration concerns. The report does not assign invented counts to individual test categories because a per-category breakdown was not provided.

Verification layer	Purpose	Evidence / expected assertion
Unit tests	Validate isolated domain logic and calculations.	Correct fuel-efficiency result, invalid input handling, state transition rules.
Mockito-based tests	Verify collaboration with declared dependencies.	A module calls lookup/publisher contracts rather than foreign repositories.
Contract tests	Validate DTO/interface assumptions between modules.	Required fields and contract semantics remain compatible.
Spring Boot integration tests	Exercise configured application components together.	Authentication, controllers, persistence, and transaction behaviour.
Flyway/schema validation	Ensure migrations and mappings are compatible.	Migrations execute; Hibernate validates rather than mutates schema.

7.2 Frontend Automated Testing

The supplied verification record reports 41 of 41 frontend tests passing using Vitest and React Testing Library. Tests should verify visible user outcomes rather than implementation details wherever possible. Appropriate areas include rendering of role-specific views, form validation, CSRF-aware request setup, telemetry subscription/update behaviour, explicit simulated-data labels, map/visual status synchronisation, and error/loading states. The exact distribution of the 41 tests was not supplied.

7.3 MySQL Persistence and Restart Verification

Restart persistence was reported verified against MySQL 8.4 LTS with zero schema drift. A reproducible verification sequence is:

14. Start MySQL 8.4 LTS and create/configure the application schema and credentials.
15. Start the backend and confirm that Flyway applies the expected migration sequence.
16. Confirm that JPA mappings pass Hibernate schema validation with `ddl-auto=validate`.
17. Create representative records through supported application workflows.
18. Stop and restart the application without dropping the database.
19. Confirm that records persist and that Flyway/schema validation reports no drift or unexpected DDL mutation.

The result is a persistence/restart claim, not a claim of disaster recovery, backup restoration, high availability, or production-scale durability testing.

7.4 Security and Penetration Testing

The project specification calls out CSRF rejection, unauthorized endpoint interception, and session-cookie attribute verification. These checks are important because authentication alone does not guarantee that a user is authorised for every action.

Security check	Test procedure	Expected result
CSRF rejection	Submit a state-changing request without a token or with an invalid token.	Request is rejected before the protected operation executes.
Authentication boundary	Request a protected endpoint without an authenticated session.	Request is denied or redirected according to the API's configured behaviour.
Role enforcement	Authenticate as a role lacking the required permission and attempt the operation.	Operation is denied; no unauthorized state change occurs.
Cookie attributes	Inspect session-cookie response headers.	`JSESSIONID` includes HttpOnly and configured SameSite policy; Secure should be enabled under HTTPS deployment.
Password storage	Inspect stored password representation and encoder configuration.	Passwords are represented by BCrypt hashes, not plaintext.
Audit behaviour	Perform a relevant administrative action and inspect the audit record.	Expected event is recorded without secret material.

The provided ground truth confirms the testing focus but does not provide a third-party penetration-test report or a complete vulnerability assessment. Therefore, this chapter describes verification controls and does not claim that an independent penetration test has been completed.

7.5 Test Summary

Area	Reported result	Interpretation
Backend automated tests	88 / 88 passing	All supplied backend automated tests passed.
Frontend automated tests	41 / 41 passing	All supplied frontend automated tests passed.
MySQL restart persistence	Verified	Persistence survived application restart in the stated environment.
Schema drift	Zero drift reported	Migration/schema validation remained consistent in the stated verification.

CHAPTER 8: CONCLUSION, HONEST DISCLOSURES & FUTURE WORK

8.1 Summary of Delivered System

FleetX demonstrates a cohesive fleet and logistics application built as a decoupled modular monolith. It integrates identity and administrative controls, fleet asset lifecycle management, driver/trip operations, shipment manifests and route-distance estimation, and telemetry-driven visualisation. The architecture uses immutable DTOs and declared lookup/publisher contracts to protect domain boundaries while retaining a single deployable Spring Boot artifact.

The data layer is governed by Flyway migrations and Hibernate schema validation against MySQL 8.4 LTS. The user interface combines React and TypeScript with Leaflet-based 2D tracking and a stylized Three.js 3D view. The supplied verification baseline—88/88 backend tests and 41/41 frontend tests passing, together with restart-persistence verification—supports the project's stated functional and integration claims within the tested environment.

8.2 Academic Disclosures

- Telemetry is simulated by `SimulatedGpsService` through software waypoint interpolation. API responses and UI elements explicitly label it "SIMULATED DATA". It is not a live feed from physical GPS hardware.

- Route distance is estimated using the Haversine great-circle formula with R = 6371 km. It is a straight-line approximation and does not represent a road-network route.
- The Three.js fleet view is a stylized procedural visualisation synchronized with telemetry state. It is not a physically surveyed digital twin.
- The stated sub-500 ms telemetry latency is a target unless accompanied by a reproducible measurement report.
- The supplied test totals are reported as given; no additional performance, penetration-test, availability, or production-scale claims are inferred.

8.3 Limitations and Bottlenecks

- Simulated telemetry cannot validate GPS hardware reliability, real-world signal loss, device calibration, or actual vehicle behaviour.
- Haversine estimates may materially understate road distance and cannot account for traffic, road restrictions, terrain, or time-dependent travel conditions.
- A single deployable monolith does not permit independent scaling or independent deployment of individual modules.
- WebSocket fan-out and UI responsiveness may become bottlenecks at larger client counts; load-test evidence is needed before defining capacity.
- External map tiles depend on network availability and provider usage rules.
- The exact physical database schema and individual test-case breakdown were not included in the project brief; those details should be appended from repository evidence for a fully reproducible submission.
- Production deployment would require operational controls such as secrets management, HTTPS configuration, backups, monitoring, structured logging, and a documented recovery procedure.

8.4 Future Enhancements

8.4.1 Physical OBD-II and Telematics Integration

A future iteration may integrate OBD-II adapters or dedicated telematics hardware to collect odometer, engine, fuel, and location readings. Such integration would require device identity, protocol adapters, timestamp validation, outlier detection, connectivity-loss handling, and a strict separation between verified device data and simulated data.

8.4.2 Mobile Companion Application

A mobile client could support drivers with assigned trip details, milestone updates, document reminders, and delivery confirmation. Mobile authentication, offline behaviour, secure token/session storage, device permissions, and privacy controls would require separate design and testing.

8.4.3 Native Road-Graph Routing

Road-network routing could replace straight-line estimates with routes that account for road geometry and restrictions. ETA computation could then include estimated speed profiles and, where permitted, traffic information. The UI and API should clearly distinguish calculated road distance from Haversine baseline distance.

8.4.4 Additional Engineering Improvements

- Define load-test scenarios for WebSocket throughput, concurrent sessions, and tail latency.
- Add automated architecture rules to prevent cross-domain repository/entity dependencies.
- Add observability with correlation IDs, metrics, structured logs, and alerting.
- Document backup/restore drills and recovery objectives.
- Introduce stronger geospatial validation and edge-case tests for polygon boundaries and coordinate wraparound.
- Evaluate whether domain growth justifies extraction into services; retain the modular monolith until operational and organizational needs justify the added distributed-systems cost.

REFERENCES

[1] Oracle, “Java SE 21 Documentation,” Oracle. [Online]. Available: https://docs.oracle.com/en/java/javase/21/. Accessed: Oct. 16, 2026.

[2] VMware, Inc., “Spring Boot Reference Documentation,” Spring. [Online]. Available: https://docs.spring.io/spring-boot/. Accessed: Oct. 16, 2026.

[3] VMware, Inc., “Spring Security Reference,” Spring. [Online]. Available: https://docs.spring.io/spring-security/reference/. Accessed: Oct. 16, 2026.

[4] VMware, Inc., “Spring Framework Reference Documentation: WebSocket Support,” Spring. [Online]. Available: https://docs.spring.io/spring-framework/reference/web/websocket.html. Accessed: Oct. 16, 2026.

[5] Oracle, “Jakarta Persistence,” Jakarta EE. [Online]. Available: https://jakarta.ee/specifications/persistence/. Accessed: Oct. 16, 2026.

[6] Oracle, “MySQL 8.4 Reference Manual,” MySQL. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/. Accessed: Oct. 16, 2026.

[7] Redgate, “Flyway Documentation,” Flyway. [Online]. Available: https://documentation.red-gate.com/flyway. Accessed: Oct. 16, 2026.

[8] Meta Open Source, “React Documentation,” React. [Online]. Available: https://react.dev/. Accessed: Oct. 16, 2026.

[9] Microsoft, “TypeScript Documentation,” TypeScript. [Online]. Available: https://www.typescriptlang.org/docs/. Accessed: Oct. 16, 2026.

[10] Vite Contributors, “Vite Guide,” Vite. [Online]. Available: https://vite.dev/guide/. Accessed: Oct. 16, 2026.

[11] Bootstrap Team, “Bootstrap 5 Documentation,” Bootstrap. [Online]. Available: https://getbootstrap.com/docs/5.3/. Accessed: Oct. 16, 2026.

[12] Leaflet Contributors, “Leaflet Documentation,” Leaflet. [Online]. Available: https://leafletjs.com/reference.html. Accessed: Oct. 16, 2026.

[13] OpenStreetMap Contributors, “Copyright and License,” OpenStreetMap. [Online]. Available: https://www.openstreetmap.org/copyright. Accessed: Oct. 16, 2026.

[14] Three.js Authors, “Three.js Documentation,” Three.js. [Online]. Available: https://threejs.org/docs/. Accessed: Oct. 16, 2026.

[15] Axios Contributors, “Axios Documentation,” GitHub. [Online]. Available: https://axios-http.com/docs/intro. Accessed: Oct. 16, 2026.

[16] Vitest Contributors, “Vitest Guide,” Vitest. [Online]. Available: https://vitest.dev/guide/. Accessed: Oct. 16, 2026.

[17] Testing Library, “React Testing Library,” Testing Library. [Online]. Available: https://testing-library.com/docs/react-testing-library/intro/. Accessed: Oct. 16, 2026.

[18] JUnit Team, “JUnit 5 User Guide,” JUnit. [Online]. Available: https://junit.org/junit5/docs/current/user-guide/. Accessed: Oct. 16, 2026.

[19] Mockito Contributors, “Mockito Documentation,” Mockito. [Online]. Available: https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html. Accessed: Oct. 16, 2026.

[20] AssertJ Team, “AssertJ Core,” AssertJ. [Online]. Available: https://assertj.github.io/doc/. Accessed: Oct. 16, 2026.

[21] M. Fowler, “MonolithFirst,” martinfowler.com, 2015. [Online]. Available: https://martinfowler.com/bliki/MonolithFirst.html. Accessed: Oct. 16, 2026.

[22] M. Fowler, “Microservices,” martinfowler.com, 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html. Accessed: Oct. 16, 2026.

[23] R. W. Sinnott, “Virtues of the Haversine,” Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[24] D. Crockford, “Cross-Site Request Forgery (CSRF) Prevention,” OWASP Foundation, OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html. Accessed: Oct. 16, 2026.

[25] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html. Accessed: Oct. 16, 2026.

[26] OWASP Foundation, “Session Management Cheat Sheet,” OWASP Cheat Sheet Series. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. Accessed: Oct. 16, 2026.